AnyBlox: Decoupling Storage Formats from Data Processing Systems

Liam Sanft
liam.sanft@mailbox.tu-dresden.de
TU Dresden
Germany, Dresden

ABSTRACT

Here comes the abstract.

1 INTRODUCTION

For decades, database systems operated as tightly coupled units in which storage format, processing logic, and data access were controlled by a single system. External systems submitted queries and received results, while everything in between remained internal concerns. With full control over the physical layout, systems could optimize both storage and processing of data freely within their own boundaries. As long as data resided within a single system, this model was sufficient [9].

However, the volume and variety of data collected has grown drastically. Data arrives continuously from a growing number of sources, in formats tailored to their respective domains rather than to any particular database system. As a result, the database landscape has evolved from a single system controlling all aspects of data management toward a heterogeneous, modular ecosystem of specialized systems, each handling a distinct part of the data pipeline and accessing a shared storage layer. Open file formats such as Apache Parquet [4] and ORC [3], stored in data lakes [1], have gained wide adoption in this context. Beyond these, data scientists increasingly need to analyze data in domain-specific encodings from fields such as high-energy physics and bioinformatics [5].

Independently, research in data compression and encoding continues to produce formats that outperform established standards in storage efficiency and query throughput. These advances rarely reach production systems, however, because each system must independently implement support for every format it wishes to read [5].

Both trends converge on the same structural problem. Codd's principle of physical data independence [2], which assumes that a system controls its own storage format and shields applications from its physical details, no longer holds in a shared storage layer. In a shared storage layer, this assumption no longer holds. The format is determined by whoever produced the data. Every system that wants to read a given format must implement its own decoder. With N systems and M formats in active use, this results in $N \times M$ independent implementations to develop and maintain [5].

This results in higher implementation cost and maintenance effort, and simultaneously increases the risk of security vulnerabilities, as each implementation may contain bugs, as illustrated by a high-severity vulnerability in PostgreSQL's [6] JSON support that went undetected for over five years [8]. This initial hurdle discourages systems from supporting new formats altogether.

The consequence is a self-reinforcing cycle: a new format is not adopted by systems because its user base is too small to justify the implementation effort, yet its user base remains small precisely because it is not widely supported. Gienieczko et al. (2025) identify this as a core structural problem: an encoding must be popular enough to justify the cost of support, but will not gain popularity without being widely supported.

This dynamic leads to format ossification: datasets continue to be produced using outdated format specifications solely to maintain compatibility with existing readers, even when more efficient encodings are available [7]. Evolution of a format is practically prevented, because any change risks breaking compatibility with the large number of systems that have independently implemented the old specification [5].

But what would be the solution for this combinatorial integration burden? An abstraction layer between storage formats and processing systems would reduce the $N \times M$ problem to $N + M$. Each system implements the interface once, and each format provides a single implementation against it. Gienieczko et al. (2025) propose exactly this and present AnyBlox as their realization.

2 ANYBLOX

A future-proof data access solution must satisfy two conceptual requirements simultaneously. First, it must introduce an abstraction layer that decouples format internals from processing systems and system internals from decoders, thereby reducing the $N \times M$ integration burden to $N + M$. Second, this abstraction must preserve direct file access, since data lakes and shared storage environments require systems to read data directly from object stores without routing it through intermediaries. These two goals are in tension: an abstraction layer must be located somewhere, and every candidate location either breaks direct access or reintroduces format-specific coupling.

Existing approaches each resolve this tension in favor of one side. Native integration preserves direct file access and achieves the best performance through tight coupling with host internals, but requires a separate decoder implementation per format per system, reproducing the $N \times M$ problem. Extension mechanisms reduce the per-format implementation effort somewhat, but each host exposes its own extension API, so a decoder written for system A cannot be reused in system B without a near-complete rewrite. Isolated extensions — such as containerized user-defined functions studied by Saur et al. or the cloud-side UDF mechanisms of Snowflake and Amazon Redshift — provide both isolation and portability, but introduce a data transfer bottleneck between the host and the isolated decoder process. Gienieczko et al. (2025) quantify this cost for run-length encoding: while the decoder itself processes data at 6.5 GB/s, the transfer to the isolated process via Arrow Flight is limited

to 1.5 GB/s, causing the transfer to account for approximately 80% of total runtime.

Given these constraints, Gienieczko et al. (2025) argue that there is only one location for the decoder that satisfies both requirements: inside the data file itself. Packaging the decoder with the data eliminates the transfer bottleneck, since decoder and data reside in the same address space, while the abstraction is preserved because the host need not implement any format-specific logic. This is the core idea behind self-decoding data and the design premise of AnyBlox.

REFERENCES

[1] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. 2021. Lakehouse: A New Generation of Open Platforms That Unify Data Warehousing and Advanced Analytics. (2021).

[2] E. F. Codd. 1970. A Relational Model of Data for Large Shared Data Banks. *Commun. ACM* 13, 6 (June 1970), 377–387. https://doi.org/10.1145/362384.362685

[3] Apache Software Foundation. 2013. Apache ORC. https://orc.apache.org/.

[4] Apache Software Foundation. 2013. Apache Parquet. https://parquet.apache.org/.

[5] Mateusz Gienieczko, Maximilian Kuschewski, Thomas Neumann, Viktor Leis, and Jana Giceva. 2025. AnyBlox: A Framework for Self-Decoding Datasets. *Proceedings of the VLDB Endowment* 18, 11 (July 2025), 4017–4031. https://doi.org/10.14778/3749646.3749672

[6] The PostgreSQL Global Development Group. 1996. PostgreSQL. https://www.postgresql.org/.

[7] Laurens Kuiper. 2025. Query Engines: Gatekeepers of the Parquet File Format. https://duckdb.org/2025/01/22/parquet-encodings.html.

[8] Red Hat, Inc. 2017. CVE-2017-15098. https://nvd.nist.gov/vuln/detail/CVE-2017-15098.

[9] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. 2020. *Database System Concepts* (seventh edition ed.). McGraw-Hill, New York, NY.